56. Node.js: Asynchronous Code and Multithreading

Builds on **#33/#34 (async/await)**. Related: **#49 (Redis)**, **#53 (Kubernetes)**, **#55 (KubeRay)**.

> **In one line:** Node is **already asynchronous**: use `async/await` for waiting (DB, files, network). For **heavy CPU work**, use **worker threads** (Piscina). To handle **more requests** on all cores, use **cluster / PM2**.

Problem	Solution	Section
Waiting for DB / API / files	`async / await`	Part A
Several independent waits	`Promise.all`	Part A
A heavy calculation freezes the server	`worker_threads` / **Piscina**	Part B
Too many requests for one process	`cluster` / **PM2**	Part B
Heavy jobs across many machines	**BullMQ + workers + KEDA**	Part C

PART A: Asynchronous Node.js

1. Node is async by design

Node runs your JavaScript on **one main thread**. It still handles thousands of users because it **never waits** for slow work. It starts the work, moves on, and comes back when the result is ready.

> **Real-life example: one waiter** 🍽
> ❌ **Sync (blocking) waiter:** takes an order → stands in the kitchen **waiting** 10 minutes → only then serves the next table. Everyone else waits.
> ✅ **Async waiter:** takes an order → hands it to the kitchen → **immediately serves other tables** → the kitchen rings a bell 🔔 when food is ready → the waiter delivers it.
> The **kitchen** = the OS + **libuv** (Node's background helpers). The **bell** = the **event loop**.

```
Request A ──► start DB query ──► (Node is free!) ──► …result ready ──► respond A
Request B ──►            start file read ──► (free!) ──► …ready ──► respond B
Request C ──►                  …
          ▲
     one thread handles thousands of requests by never waiting
```

How the event loop works (simple)

```
┌──────────────── Your JS (main thread) ────────────────┐
│  runs code → hits await db.query() → hands it off ─┐  │
└────────────────────────────────────────────────────┼──┘
                                                     ▼
            libuv / OS does the slow I/O in the background
                                                     │
┌──────────────── Event loop ◄──── "query done!" ────┘
│  puts the callback in the queue → main thread runs it when free
└──────────────────────────────────────────────────────
```

2. Sync vs async: see the difference

```
const fs = require('fs');

// ❌ SYNC: blocks everything until the file is read
const data = fs.readFileSync('big.txt', 'utf8');
console.log('done reading');
console.log('next line');
// Output: done reading → next line

// ✅ ASYNC: starts reading, continues immediately
fs.promises.readFile('big.txt', 'utf8').then(() => console.log('done reading'));
console.log('next line');
// Output: next line → done reading
```

3. Three ways to write async code

```
// 1️⃣ Callbacks (old): nesting leads to "callback hell"
fs.readFile('a.txt', 'utf8', (err, data) => {
  if (err) return console.error(err);
  console.log(data);
});

// 2️⃣ Promises: .then / .catch
fs.promises.readFile('a.txt', 'utf8')
  .then((data) => console.log(data))
  .catch((err) => console.error(err));

// 3️⃣ async / await (modern ✅): looks like normal code, but doesn't block
```

```
async function read() {
  try {
    const data = await fs.promises.readFile('a.txt', 'utf8');
    console.log(data);
  } catch (err) {
    console.error(err);
  }
}
```

> `await` pauses **only this function**, not the whole server. Node serves other requests in the meantime.

4. In NestJS

```
@Get(':id')
async findOne(@Param('id', ParseIntPipe) id: number) {
  return this.studentService.findOne(id);      // returns a Promise → Nest awaits it
}

// service
async findOne(id: number) {
  return this.repo.findOneBy({ id });          // DB query runs in background ✅
}
```

TypeORM, Mongoose, Redis (`cache.get`), `fetch` and the AWS S3 SDK are **all async already**. You just `await` them.

5. Run things in parallel with `Promise.all`

```
// ❌ Sequential: 3 queries one after another = 300 ms
const user   = await getUser(id);      // 100 ms
const orders = await getOrders(id);    // 100 ms
const cart   = await getCart(id);      // 100 ms

// ✅ Parallel: all 3 at once = ~100 ms 🚀
const [user, orders, cart] = await Promise.all([
  getUser(id),
  getOrders(id),
  getCart(id),
]);
```

Helper	Use
`Promise.all([...])`	All must succeed; run together. One fails, so the whole call fails.
`Promise.allSettled([...])`	Run all and get each result, even if some fail
`Promise.race([...])`	The first one to finish (success or fail) wins, e.g. for timeouts
`Promise.any([...])`	The first one to **succeed** wins

6. What BLOCKS Node (avoid these ✕)

Async only helps with **waiting** (I/O). **Heavy CPU work blocks the single thread**, and then the whole server freezes.

Blocking ✕	Use instead ✓
`fs.readFileSync`, `fs.writeFileSync` in request handlers	`fs.promises.readFile`
Big loops (counting primes, millions of iterations)	**worker_threads / Piscina** (Part B)
`JSON.parse` on a huge (100 MB) string	Streaming parsers
`bcrypt.hashSync`, `crypto.pbkdf2Sync`	`bcrypt.hash` (async)
Image resizing in pure JS on the main thread	`sharp` (uses background threads) or a worker
`while (Date.now() < end) {}` (busy waiting)	`await new Promise(r => setTimeout(r, ms))`

```
// ✕ This freezes ALL users for ~5 seconds
app.get('/heavy', (req, res) => {
  let count = 0;
  for (let i = 0; i < 5e9; i++) count++;
  res.send({ count });
});
// While it runs, even GET /health won't answer!
```

7. Common async mistakes

```
// ✕ Forgot await → you get a Promise, not the data
const user = this.repo.findOneBy({ id });     // Promise { <pending> }
```

```
// ✕ await inside a for-loop → runs one by one (slow)
for (const id of ids) await this.process(id);

// ✓ parallel (for very large lists, process in batches)
await Promise.all(ids.map((id) => this.process(id)));

// ✕ no try/catch and no .catch → unhandled rejection
// ✓ use try/catch, or let NestJS exception filters handle it
```

PART B: Multithreading in Node.js

8. Why we need threads

`async/await` does **not** fix CPU-heavy work, because the calculation itself still runs on the one main thread. To truly run JavaScript **in parallel**, you need **more threads**.

Way	What it gives you	Use for
1. `worker_threads` (built-in)	Real extra threads inside one Node process	**CPU-heavy work**
2. Piscina (npm)	A thread **pool**: creates threads once, reuses them, queues work ✓	The same, easier and safer in real apps
3. `cluster` **/ PM2**	Several **processes** (app copies), one per core	Handling **more HTTP requests**

> **Your laptop:** the Intel i3-1215U has **6 cores / 8 threads**, so up to about 8 things can run truly in parallel. Check it in code with `require('os').availableParallelism()`.

9. `worker_threads` (built-in, no install)

worker.js: code that runs in the new thread

```
const { parentPort, workerData } = require('worker_threads');

function countPrimes(limit) {
  let count = 0;
  for (let n = 2; n < limit; n++) {
    let prime = true;
    for (let d = 2; d * d <= n; d++) if (n % d === 0) { prime = false; break; }
    if (prime) count++;
  }
  return count;
```

```
}

parentPort.postMessage(countPrimes(workerData.limit));   // send result back
```

main.js: starts the workers

```
const { Worker } = require('worker_threads');

function runWorker(limit) {
  return new Promise((resolve, reject) => {
    const worker = new Worker('./worker.js', { workerData: { limit } });
    worker.on('message', resolve);
    worker.on('error', reject);
    worker.on('exit', (code) => code !== 0 && reject(new Error(`exit ${code}`)));
  });
}

(async () => {
  console.time('4 threads');
  const results = await Promise.all([
    runWorker(5_000_000), runWorker(5_000_000),
    runWorker(5_000_000), runWorker(5_000_000),
  ]);                                    // 4 threads run IN PARALLEL 🚀
  console.timeEnd('4 threads');
  console.log(results);
})();
```

```
Main thread ──► creates 4 workers ──► stays FREE for other work
   ├── Worker 1 (core 1) ── countPrimes ──┐
   ├── Worker 2 (core 2) ── countPrimes ──┤
   ├── Worker 3 (core 3) ── countPrimes ──┼──► postMessage(results) ──► main
   └── Worker 4 (core 4) ── countPrimes ──┘
```

Rules:

• Workers **don't share variables** with the main thread. Data is **copied** through `workerData` and `postMessage` (except `SharedArrayBuffer`).

• Creating a worker costs tens of milliseconds plus memory, so **don't create one per request**. Use a **pool** (Piscina).

10. Piscina: a thread pool in NestJS ✓

```
npm install piscina
```

primes.worker.js

```
module.exports = ({ limit }) => {
  let count = 0;
  for (let n = 2; n < limit; n++) {
    let prime = true;
    for (let d = 2; d * d <= n; d++) if (n % d === 0) { prime = false; break; }
    if (prime) count++;
  }
  return count;            // just return, and Piscina sends it back
};
```

primes.service.ts

```
import { Injectable, OnModuleDestroy } from '@nestjs/common';
import Piscina from 'piscina';
import { resolve } from 'path';
import { availableParallelism } from 'os';

@Injectable()
export class PrimesService implements OnModuleDestroy {
  private pool = new Piscina({
    filename: resolve(__dirname, 'primes.worker.js'),   // compiled .js in dist/
    maxThreads: Math.max(1, availableParallelism() - 1), // leave 1 core for main thread
  });

  count(limit: number): Promise<number> {
    return this.pool.run({ limit });     // runs on a pool thread, main stays free ✅
  }

  async onModuleDestroy() {
    await this.pool.destroy();
  }
}
```

primes.controller.ts

```
@Get(':limit')
count(@Param('limit', ParseIntPipe) limit: number) {
  return this.primesService.count(limit);
}
```

> Now `GET /primes/10000000` runs on a background thread, and **other users' requests keep working** ✅.

> **The worker file must be `.js` at runtime.** In NestJS, either write it as `.ts` and point `filename` to the compiled file in `dist/`, or keep a `.js` file and add it to `assets` in `nest-cli.json` so it's copied into `dist/`.

11. `cluster` / PM2: multiple processes

This doesn't speed up one heavy task. It runs **several copies of your server**, so more requests are handled at once.

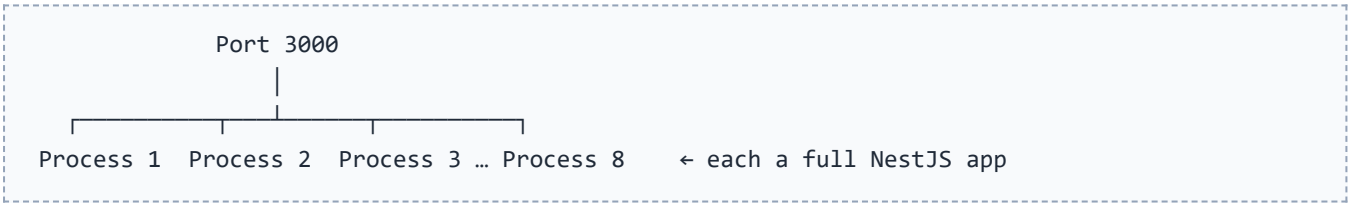
Built-in `cluster`

```
const cluster = require('cluster');
const { availableParallelism } = require('os');

if (cluster.isPrimary) {
  for (let i = 0; i < availableParallelism(); i++) cluster.fork();   // e.g. 8 copies
  cluster.on('exit', () => cluster.fork());                          // restart if one dies
} else {
  require('./dist/main');      // each copy runs your NestJS app on the SAME port
}
```

Easier: PM2 ✓

```
npm install -g pm2
pm2 start dist/main.js -i max      # one process per CPU core
pm2 list
pm2 logs
```



> Each process has its **own memory**, so in-memory caches and sessions aren't shared. Use **Redis** (#49) for shared data. In Docker or Kubernetes, you usually run **1 process per container** and scale **pods** instead of using cluster.

12. Threads vs processes

	`worker_threads` / Piscina	`cluster` / PM2
Type	Threads inside **one** process	Many separate **processes**
Memory	Lighter	Each copy uses full app memory
Solves	One **CPU-heavy task** blocking the server	**More requests** at the same time
Communication	`postMessage` / `SharedArrayBuffer`	None directly (separate apps)
Example	Resize image, count primes, parse a big CSV	A busy API on an 8-core server

13. Bonus: Node already uses threads behind the scenes

Some built-in work secretly runs on **libuv's thread pool** (**4 threads by default**): `fs.*`, `crypto.pbkdf2`, `scrypt`, async `bcrypt`, `zlib`, `dns.lookup`.

```
# increase it if you hash many passwords or read many files at once
UV_THREADPOOL_SIZE=8 node dist/main.js
```

> So "Node is single-threaded" really means **your JavaScript** runs on one main thread. Node itself uses extra threads for I/O.

PART C: Beyond one machine

14. Threads vs KubeRay vs BullMQ

Scale	Python	Node.js
One machine, all cores	`multiprocessing` / Ray (local)	`worker_threads` **/ Piscina**
Many machines, autoscaling	**Ray + KubeRay** (#55)	**BullMQ + worker pods + KEDA**

> **Kitchen example** 🍳
> **worker_threads** = one kitchen with 8 cooks. It's faster, but if the kitchen burns down, everything stops.
> **KubeRay / BullMQ + KEDA** = a chain of many kitchens, with a manager who opens more at rush hour and closes them when it's quiet.

```
BullMQ queue (Redis)
   ├── Pod 1 → 4 worker threads (Piscina)
   ├── Pod 2 → 4 worker threads
   └── Pod N → 4 worker threads      ← KEDA adds pods when the queue grows
```

15. When to use what

Your situation	Use
Waiting on DB, API or files	`async/await`, no threads needed
Several independent waits	`Promise.all`
A heavy calculation freezes the server	**Piscina /** `worker_threads`
Too many requests for one process	**PM2** `-i max` or more pods
Background jobs (emails, reports, images)	**BullMQ**
Heavy jobs across many machines	**BullMQ + KEDA** on Kubernetes
Heavy ML / AI	A Python service (Ray Serve), called from NestJS over HTTP

16. Quick summary

Question	Answer
Is Node async?	**Yes, by default** for I/O (DB, files, network, timers)
How do I write async code?	`async/await` (modern), or Promises
How does it work?	One main thread + the **event loop**; slow I/O runs in the background (libuv)
What blocks Node?	`*Sync` functions and **heavy CPU loops**
Is Node single-threaded?	Your JS runs on one main thread by default, but Node can use more
How do I add threads?	`worker_threads` or **Piscina** (a pool, recommended)
How do I use all cores for requests?	`cluster` or **PM2** `-i max`
How many threads?	About your CPU threads minus 1 (your laptop: 8 → 7)
Do threads share variables?	No, data is copied with `postMessage` (except `SharedArrayBuffer`)
Many machines?	**BullMQ + KEDA** (the Node version of KubeRay)